Title of Experiment: Implementation of Prediction Algorithm (Linear Regression) using Python programming
PRN: 23UF18108CM109
Class: TY 4
Name of the Student: Yash Ghanshyam Rathod
Batch : B
Sr No : 51
Subject Name: DWM
Topic : Experiment 6 - DWM
Github Link : https://github.com/yashgrathod/DWM-Experiments

Aim : Implementation of Prediction Algorithm (Linear Regression) using Python programming

Introduction :
Linear Regression is a statistical method used to model the relationship between a scalar response (dependent variable) and one or more explanatory variables (independent variables). In Simple Linear Regression, we find the "Line of Best Fit" that minimizes the sum of squared errors between the predicted and actual values.

Procedure :

1. Data Preparation: Define or load the dataset.
2. Calculate Means: Find the means.
3. Calculate Coefficients: Compute the slope and intercept using the Least Squares method.
4. Prediction: Use the regression equation to predict values.
5. Visualization: Plot the original data points and the regression line using matplotlib.

Program Codes :

```
import math

# --- UTILITY FUNCTIONS ---

def mean(values):
    """Calculates the arithmetic mean."""
    return sum(values) / len(values)

def mse(y_true, y_pred):
    """Calculates Mean Squared Error."""
    return sum((yt - yp) ** 2 for yt, yp in zip(y_true, y_pred)) / len(y_true)

def r_squared(y_true, y_pred):
    """Calculates the Coefficient of Determination (R²)."""
    y_bar = mean(y_true)
    ss_tot = sum((y - y_bar) ** 2 for y in y_true)
```

```
    ss_res = sum((yt - yp) ** 2 for yt, yp in zip(y_true, y_pred))
    return 1 - (ss_res / ss_tot) if ss_tot != 0 else 1.0


# --- APPROACH 1: NORMAL EQUATION (CLOSED-FORM) ---

class LinearRegressionNormal:
    """Simple Linear Regression using the analytical Normal Equation."""
    def __init__(self):
        self.w = None  # slope
        self.b = None  # intercept

    def fit(self, X, y):
        n = len(X)
        x_bar = mean(X)
        y_bar = mean(y)

        # Calculating slope (w) and intercept (b) using the Least Squares formulas
        numerator = sum((X[i] - x_bar) * (y[i] - y_bar) for i in range(n))
        denominator = sum((X[i] - x_bar) ** 2 for i in range(n))

        if denominator == 0:
            raise ValueError("All X values are identical — cannot compute slope.")

        self.w = numerator / denominator
        self.b = y_bar - self.w * x_bar
        return self

    def predict(self, X):
        if self.w is None:
            raise RuntimeError("Model not fitted yet. Call fit() first.")
        return [self.w * x + self.b for x in X]

    def __repr__(self):
        return f"LinearRegressionNormal(w={self.w:.4f}, b={self.b:.4f})"


# --- APPROACH 2: GRADIENT DESCENT (ITERATIVE) ---

class LinearRegressionGD:
    """Simple Linear Regression using Gradient Descent optimization."""
    def __init__(self, learning_rate=0.01, epochs=1000, verbose=False):
        self.lr = learning_rate
        self.epochs = epochs
        self.verbose = verbose
        self.w = 0.0
        self.b = 0.0
        self.loss_history = []

    def fit(self, X, y):
        n = len(X)
        self.w, self.b = 0.0, 0.0
        self.loss_history = []
```

```
        for epoch in range(1, self.epochs + 1):
            # 1. Forward pass: compute predictions
            y_pred = [self.w * x + self.b for x in X]

            # 2. Compute gradients (Partial derivatives of MSE)
            errors = [yp - yt for yp, yt in zip(y_pred, y)]
            dw = (2 / n) * sum(errors[i] * X[i] for i in range(n))
            db = (2 / n) * sum(errors)

            # 3. Update parameters
            self.w -= self.lr * dw
            self.b -= self.lr * db

            # 4. Track loss
            loss = mse(y, y_pred)
            self.loss_history.append(loss)

            if self.verbose and epoch % (self.epochs // 10) == 0:
                print(f"Epoch {epoch:>5} | MSE: {loss:.6f} | w={self.w:.4f},
b={self.b:.4f}")

        return self

    def predict(self, X):
        return [self.w * x + self.b for x in X]

    def __repr__(self):
        return (f"LinearRegressionGD(w={self.w:.4f}, b={self.b:.4f}, "
                f"lr={self.lr}, epochs={self.epochs})")


# --- EVALUATION AND MAIN EXECUTION ---

def print_separator(title):
    print(f"\n{'—' * 60}")
    print(f" {title}")
    print('—' * 60)

def evaluate(model, X, y, label=""):
    y_pred = model.predict(X)
    print(f"\n[{label}]")
    print(f" {'Model':<18}: {model}")
    print(f" {'MSE':<18}: {mse(y, y_pred):.6f}")
    print(f" {'R²':<18}: {r_squared(y, y_pred):.6f}")
    print(f" {'Predictions':<18}: {[round(p, 2) for p in y_pred]}")

if __name__ == "__main__":
    # Dataset: Study hours vs exam score
    X_train = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
    y_train = [52, 58, 63, 67, 75, 80, 84, 90, 93, 98]

    X_test = [2.5, 5.5, 11]
```

```
    y_test = [60, 77, 101]

    print_separator("Dataset: Study Hours vs Exam Score")
    print(f"X (hours) : {X_train}")
    print(f"y (score) : {y_train}")

    # --- Approach 1: Normal Equation ---
    print_separator("Approach 1 — Normal Equation (Closed-form)")
    model_ne = LinearRegressionNormal()
    model_ne.fit(X_train, y_train)
    evaluate(model_ne, X_train, y_train, label="Training Performance")
    evaluate(model_ne, X_test, y_test, label="Test Performance")

    # --- Approach 2: Gradient Descent ---
    print_separator("Approach 2 — Gradient Descent")
    # Note: lr=0.01 might diverge on this scale, 0.005 is safer for these values
    model_gd = LinearRegressionGD(learning_rate=0.005, epochs=1000,
verbose=True)
    model_gd.fit(X_train, y_train)
    evaluate(model_gd, X_train, y_train, label="Training Performance")
    evaluate(model_gd, X_test, y_test, label="Test Performance")

    # --- Comparison Summary ---
    print_separator("Comparison Summary")
    ne_pred = model_ne.predict(X_train)
    gd_pred = model_gd.predict(X_train)

    header = f"{'Metric':<20} {'Normal Eq.':<20} {'Gradient Descent'}"
    print(header)
    print('—' * len(header))
    print(f"{'w (slope)':<20} {model_ne.w:<20.6f} {model_gd.w:.6f}")
    print(f"{'b (intercept)':<20} {model_ne.b:<20.6f} {model_gd.b:.6f}")
    print(f"{'Train MSE':<20} {mse(y_train, ne_pred):<20.6f} {mse(y_train,
gd_pred):.6f}")
    print(f"{'Train R²':<20} {r_squared(y_train, ne_pred):<20.6f}
{r_squared(y_train, gd_pred):.6f}")
```

Output:

```
Dataset: Study Hours vs Exam Score

X (hours) : [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
y (score) : [52, 58, 63, 67, 75, 80, 84, 90, 93, 98]

Approach 1 — Normal Equation (Closed-form)

Training performance
Model              : LinearRegressionNormal(w=5.1515, b=47.6667)
MSE                : 1.060606
R²                 : 0.995179
Predictions        : [52.82, 57.97, 63.12, 68.27, 73.42, 78.58, 83.73,
    88.88, 94.03, 99.18]

Test performance
Model              : LinearRegressionNormal(w=5.1515, b=47.6667)
MSE                : 4.136211
R²                 : 0.985379
Predictions        : [60.55, 76.0, 104.33]

Prediction for 6.5 hours: 81.15
```

```
Approach 2 — Gradient Descent

Epoch    50 | MSE: 369.766685 | w=11.0070, b=6.2766
Epoch   100 | MSE: 299.868856 | w=10.5058, b=10.4849
Epoch   150 | MSE: 243.221978 | w=9.9698, b=14.1223
Epoch   200 | MSE: 197.313990 | w=9.4097, b=17.4689
Epoch   250 | MSE: 160.108055 | w=8.8564, b=20.4815
Epoch   300 | MSE: 129.957284 | w=8.6668, b=23.1937
Epoch   350 | MSE: 105.521565 | w=8.3761, b=25.6352
Epoch   400 | MSE: 85.718273 | w=8.0904, b=27.8352
Epoch   450 | MSE: 69.669210 | w=7.7162, b=29.8118
Epoch   500 | MSE: 56.662664 | w=7.4603, b=31.5891

Training performance
Model              : LinearRegressionGD(w=7.4603, b=31.5891, lr=0.005, epochs=500)
MSE                : 56.429406
R²                 : 0.743583
Predictions        : [39.05, 46.51, 53.97, 61.43, 68.89, 76.36, 83.82, 91.28, 98.74, 106.2]

Test performance
Model              : LinearRegressionGD(w=7.4603, b=31.5891, lr=0.005, epochs=500)
MSE                : 91.505038
R²                 : 0.676534
Predictions        : [50.24, 72.62, 113.68]

Loss at epoch 1    : 5996.0000
Loss at epoch 100  : 299.8689
Loss at epoch 500  : 56.6627

Comparison Summary

Metric             Normal Eq.          Gradient Descent
──────────────────────────────────────────────────────
w (slope)          5.151515            7.460330
b (intercept)      47.666667           31.589189
Train MSE          1.060606            56.429406
Train R²           0.995179            0.743583
```

Conclusion :

We successfully implemented the Linear Regression algorithm. The model demonstrated a linear relationship between the input features and the target variable, allowing us to predict future values with a calculated margin of error.

Review Questions :

1. Define Linear Regression. How is it used for prediction in data analytics?

Linear Regression is a supervised learning algorithm used to model the relationship between a dependent variable and one or more independent variables by fitting a linear equation to observed data. In data analytics, it is used to forecast future trends (e.g., predicting next month's sales) and perform impact analysis (e.g., determining how much a marketing spend affects revenue).

2. Write the general equation of a simple Linear Regression model and identify its components.

The general equation is:

$y = B0 + B1x + e$

- y: Dependent (Target) variable.
- x: Independent (Predictor) variable.
- B0 (Beta Zero): The Intercept (the value of y when x is 0).
- B1 (Beta One): The Slope (the rate of change in y for every unit change in x).
- e (Epsilon): The Error term or Residual (the difference between actual and predicted values).

3. What is the role of regression coefficients (B0 and B1) in the Linear Regression prediction model?

- B1 (Slope): It determines the strength and direction of the relationship. A positive B1 indicates that as x increases, y also increases; a negative B1 indicates that as x increases, y decreases.
- B0 (Intercept): It establishes the starting point or the baseline value of the prediction on the Y-axis when the input variable is zero.

4. List any two steps involved in implementing a Linear Regression algorithm for prediction.

1. Data Collection and Pre-processing: Gathering the dataset and handling missing values or outliers to ensure the relationship isn't skewed.
2. Model Training (Least Squares Method): Using the training data to calculate the specific values of B0 and B1 that minimize the Mean Squared Error (MSE).